

# Discovery Piscine

## Module 8 - Python

*Summary: In this Module, we see how to use functions and scopes.*

*Version: 2.1*

# Contents


| | | |
|---|---|---|
| I | A word about this Discovery Piscine | 2 |
| II | Introduction | 3 |
| III | General instructions | 4 |
| IV | Exercise 00: Discovering Functions! | 5 |
| V | Exercise 01: upcase_it | 6 |
| VI | Exercise 02: downcase_all | 7 |
| VII | Exercise 03: greetings_for_all | 8 |
| VIII | Exercise 04: functions_everywhere | 10 |
| IX | Exercise 05: scope_that | 12 |
| X | Submission and peer-evaluation | 13 |

# Chapter I

# A word about this Discovery Piscine

Welcome!
You will begin a Module of this Discovery Piscine of computer programming. Our goal is to introduce you to the code behind the software you use daily and immerse you in peer learning, the educational model of 42.

Programming is about logic, not mathematics. It gives you basic building blocks that you can assemble in countless ways. There is no single "correct" solution to a problem—your solution will be unique, just as each of your peers' solutions will be.

Fast or slow, elegant or messy, as long as it works, that's what matters! These building blocks will form a sequence of instructions (for calculations, displays, etc.) that the computer will execute in the order you design.

Instead of providing you with a course where each problem has only one solution, we place you in a peer-learning environment. You'll search for elements that could help you tackle your challenge, refine them through testing and experimentation, and ultimately create your own program. Discuss with others, share your perspectives, come up with new ideas together, and test everything yourself to ensure it works.

Peer evaluation is a key opportunity to discover alternative approaches and spot potential issues in your program that you may have missed (consider how frustrating a program crash can be). Each reviewer will approach your work differently—like clients with varying expectations—giving you fresh perspectives. You may even form connections for future collaborations.

By the end of this Piscine, your journey will be unique. You will have tackled different challenges, validated different projects, and chosen different paths than others—and that's perfectly fine! This is both a collective and individual experience, and everyone will gain something from it.

Good luck to all; we hope you enjoy this journey of discovery.

# Chapter II

# Introduction

What this Module will show you:

- You will learn how to define and call functions.
- You will learn about variable scopes in Python.

# Chapter III

# General instructions

Unless otherwise specified, the following rules apply every day of this Piscine.

- This document is the only trusted source. Do not rely on rumors.
- This document may be updated up to one hour before the submission deadline.
- Assignments must be completed in the specified order. Later assignments will not be evaluated unless all previous ones are completed correctly.
- Pay close attention to the access rights of your files and folders.
- Your assignments will be evaluated by your fellow Piscine peers.
- All shell assignments must run using `/bin/bash`.
- You must not leave any file in your submission workspace other than those explicitly requested by the assignments.
- Have a question? Ask your neighbor on your left. If not, try your neighbor on your right.
- Every technical answer you need can be found in the `man` pages or online.
- Remember to use the Piscine forum of your intranet and Slack!
- Read the examples thoroughly, as they may reveal requirements that aren't immediately obvious in the assignment description.
- By Thor, by Odin! Use your brain!!!

# Chapter IV

# Exercise 00: Discovering Functions!

|  | Exercise00 |
|---|---|
| Discovering Functions! | |
| Directory: *ex*00/ | |
| Files to Submit: `hello_all.py` | |
| Authorized: `All` | |

- Create a program called `hello_all.py`.
- Ensure this program is executable.
- This program should:
  - Define a function called `hello` that prints "Hello, everyone!".
  - Call this function to display the message. Refer to the example below, except that the function definition has been hidden.

```
?> cat hello_all.py
# Your function definition

hello()
?> ./hello_all.py
Hello, everyone!
?>
```





`Search for "function definition in Python".`

# Chapter V

# Exercise 01: upcase_it

|  | Exercise01 |
|---|---|
| The Return of upcase! | |
| Directory: *ex*01/ | |
| Files to Submit: `upcase_it.py` | |
| Authorized: `All` | |

- Create a program called `upcase_it.py` (again!)
- Ensure this program is executable.
- Define a function in the program named `upcase_it`.
- The `upcase_it` function should take a string as an argument and return the string in uppercase.
- Test the function by calling it in your program. In the example below, we test it with "hello":

```
?> cat upcase_it.py
# Your function definition

print(upcase_it("hello"))
?> ./upcase_it.py
HELLO
?>
```



# Chapter VI

# Exercise 02: downcase_all

|  | Exercise02 |
|---|---|
| Let's Stroll in the Array | |
| Directory: *ex*02/ | |
| Files to Submit: `downcase_all.py` | |
| Authorized: `All` | |

- Create a program called `downcase_all.py`.
- Ensure this program is executable.
- Define a function in this program called `downcase_it`.
- The `downcase_it` function should take a string as an argument and return the string in lowercase.
- Apply this function to each program's parameters and display the return value for each.
- If there are no parameters, display "none" followed by a line break.

```
?> ./downcase_all.py
none
?> ./downcase_all.py "HELLO WORLD" "I understood Arrays well!"
hello world
i understood arrays well!
?>
```

# Chapter VII

# Exercise 03: greetings_for_all

|  | Exercise03 |
|---|---|
| Let's Say Hello to everyone! | |
| Directory: *ex*03/ | |
| Files to Submit: `greetings_for_all.py` | |
| Authorized: `All` | |

- Create a program called `greetings_for_all.py` that does not take any parameters.
- Ensure this program is executable.
- Create a function called `greetings` which takes a name as a parameter and displays a welcome message with that name.
- If the function is called without an argument, its default parameter should be "noble stranger".
- If the function is called with an argument that is not a string, an error message should be displayed instead of the welcome message.

```
?> cat greetings_for_all.py | cat -e
# your function definition here

greetings('Alexandra')
greetings('Wil')
greetings()
greetings(42)
```

will produce the following output:



```
?> ./greetings_for_all.py | cat -e
Hello, Alexandra.$
Hello, Wil.$
Hello, noble stranger.$
Error! It was not a name.$
?>
```



`Google default parameter, isinstance function.`

# Chapter VIII

# Exercise 04: functions_everywhere

|  | Exercise04 |
|---|---|
| Functions Everywhere! | |
| Directory: *ex*04/ | |
| Files to Submit: `functions_everywhere.py` | |
| Authorized: `All` | |

- Create a program called `functions_everywhere.py` that takes parameters.
- Ensure this program is executable.
- You need to create two different functions in this program:
  - The `shrink` function: should take a string as a parameter and display the first eight characters of that string.
  - The `enlarge` function: should take a string as a parameter and append 'Z' characters until the string has a total of eight characters. Then, it displays the resulting string.
- If there are no parameters, display "none" followed by a line break.
- For each argument passed to the program:
  - If the argument has more than eight characters, call the `shrink` function on it.
  - If the argument has fewer than eight characters, call the `enlarge` function on it.
  - If the argument has exactly eight characters, display it directly followed by a new line.

```
?> ./functions_everywhere.py | cat -e
none$
?> ./functions_everywhere.py 'lol' 'physically' 'backpack' | cat -e
lolZZZZZ$
```

```
physical$
backpack$
?>
```



shrink function: Use slices.



enlarge function: Similar to arrays, you can add characters to a string using the concatenation operator.



# Chapter IX

# Exercise 05: scope_that

|  | Exercise05 |
|---|---|
| Can't touch this! | |
| Directory: *ex*05/ | |
| Files to Submit: `scope_that.py` | |
| Authorized: `All` | |

- Create a program called `scope_that.py` that does not take any parameters.
- Ensure this program is executable.
- Inside the program, create a function called `add_one` that takes a parameter and adds 1 to the parameter.
- Initialize a variable in the body of the program, display it, and then call the function that adds 1.
- Display your variable again in the body of the program.
- What do you observe?



# Chapter X

# Submission and peer-evaluation

- You must have `discovery_piscine` folder at the root of your home directory.
- Inside the `discovery_piscine` folder, you must have a folder named `module8`.
- Inside the `module8` folder, you must have a folder for each exercise.
- Exercise 00 must be in the `ex00` folder, Exercise 01 in the `ex01` folder, etc.
- Each exercise folder must contain the files requested in the assignment.



`Please note, during your defense anything that is not present in the folder for the module will not be checked.`